Структура бота «Грызлик»

Грызлик — чат-бот в MAX: кидаешь ему тему или текст параграфа, он делает тест, находит, что западает, объясняет это заново и растёт от твоих знаний. Здесь описано, как он устроен внутри: что хранит, какие экраны проходит ученик и как считает уровни. Код — в `bot/`, вся логика в одном файле `bot.py`.

1. Что бот помнит об ученике

Всё лежит в SQLite (`data/grizlik.db`), таблица `users`:

Поле	Тип	Начальное	Что хранит
`user_id`	число	—	ID ученика в MAX (первичный ключ)
`chat_id`	число	—	ID диалога, куда слать сообщения
`class`	число	0	Класс ученика (спрашиваем один раз)
`xp`	число	0	Опыт: верный ответ +10, после объяснения +5, закрытая тема +30
`level`	число	1	1 яйцо · 2 малыш · 3 подросток · 4 взрослый · 5 мудрец
`hunger`	число	100	Сытость 0–100, −30 за день без кормёжки
`streak`	число	0	Серия дней подряд
`last_feed`	текст	«»	Дата последней кормёжки (ГГГГ-ММ-ДД)
`weak`	JSON	`[]`	Слабые подтемы: `{topic, subtopic, added, r}`; `r` — сколько раз предлагали повтор
`state`	JSON	`{}`	Текущий экран/режим: `class`, `feed_input`, `quiz`, `after`, `explain`
`last_remind`	текст	«»	Дата последнего напоминания (чтобы слать раз в день)
`closed`	JSON	`[]`	Закрытые темы (для «Моего прогресса»)

Пороги уровней: 2 — 150 · 3 — 450 · 4 — 900 · 5 — 1800 опыта.

Рядом две вспомогательные таблицы:

- `seen` — ключи уже обработанных апдейтов (`msg:{mid}`, `cb:{callback_id}`). MAX иногда доставляет апдейт повторно; без этой таблицы опыт начислялся бы дважды.
- `uploads` — кэш токенов загруженных картинок, чтобы каждая стадия питомца загружалась в MAX один раз, а не при каждом сообщении.

2. Экраны и переходы

```
[START / bot_started]
  └─ class == 0 → [ONBOARDING] (с mascot.png)   иначе → [HOME]

[ONBOARDING]
  «Привет! Я Грызлик — питаюсь знаниями 🐹 В каком ты классе?»
  Кнопки: 5 6 7 8 / 9 10 11 → class = N → [FEED_INPUT]

[HOME]  — меню: картинка pet_{level}.png + статус
  «🐹 {имя уровня} · уровень N · Сытость X% · Серия Y дн. · Опыт Z · Слабые места: …»
  Кнопки:
    🍽 Покормить (новая тема) → [FEED_INPUT]
    🔁 Повторить слабое       → [GENERATE] с акцентом на первую слабую подтему
    📊 Слабые места           → [WEAK_LIST]   (видна, если weak не пуст)
    📈 Мой прогресс           → [STATS]
    ⚙️ Сменить класс          → [ONBOARDING]

[FEED_INPUT]
  «Напиши тему (например «Дроби» или «Смутное время») или вставь текст параграфа»
  Ждём текст:
    пустой/бессмысленный («ааа», «123») → вежливая просьба уточнить
    > 6000 символов → обрезать и предупредить
    длина > 300 символов → source_text = текст (ИИ отвечает только по тексту)
  → [GENERATE]

[GENERATE]  — промт quiz, user = «Класс: N · Ввод ученика: …»
  «Ням… жую параграф, секунду 🍃»
  ИИ вернул не-JSON / таймаут / 429 → «Грызлик подавился 😵» + кнопка «🔄 Попробовать ещё раз»
  Нормализация: ровно 3 варианта, корректный индекс правильного, 3–5 вопросов
  → [QUESTION]

[QUESTION]  — цикл по вопросам (обычно 5)
  «Вопрос i из N» + 3 кнопки-варианта
  Верно → «Ням! Вкусно!»; неверно → «Вкусная ошибка!» + правильный ответ
  Клавиатура убирается у исходного сообщения (повторное нажатие невозможно)
  i < N → [QUESTION], иначе → [DIAGNOSIS]

[DIAGNOSIS]
  Итог: X/N · Знаешь: … · Западает: … · +X*10 опыта · серия Y дн.
  Слабые подтемы добавляются в weak (без дублей)
  Если уровень вырос → pet_{level}.png + «🥕 Я вырос!»
  Кнопки: «Разобрать «{подтема}»» (до 3 шт.) → [EXPLAIN_MENU], «В меню»

[EXPLAIN_MENU]
  «Как объяснить «{подтема}»?»
  Кнопки: Проще | На примере | По-другому → mode = simple/example/another → [EXPLAIN]

[EXPLAIN]  — промт explain
  {explanation} + «🤔 Проверь себя: {check_question}»
  Кнопки: «Понял, проверь меня» → [FOLLOWUP] · «Объясни иначе» → [EXPLAIN_MENU] · «В меню»
  Сбой ИИ → «Попробуй ещё разок» + кнопка повтора

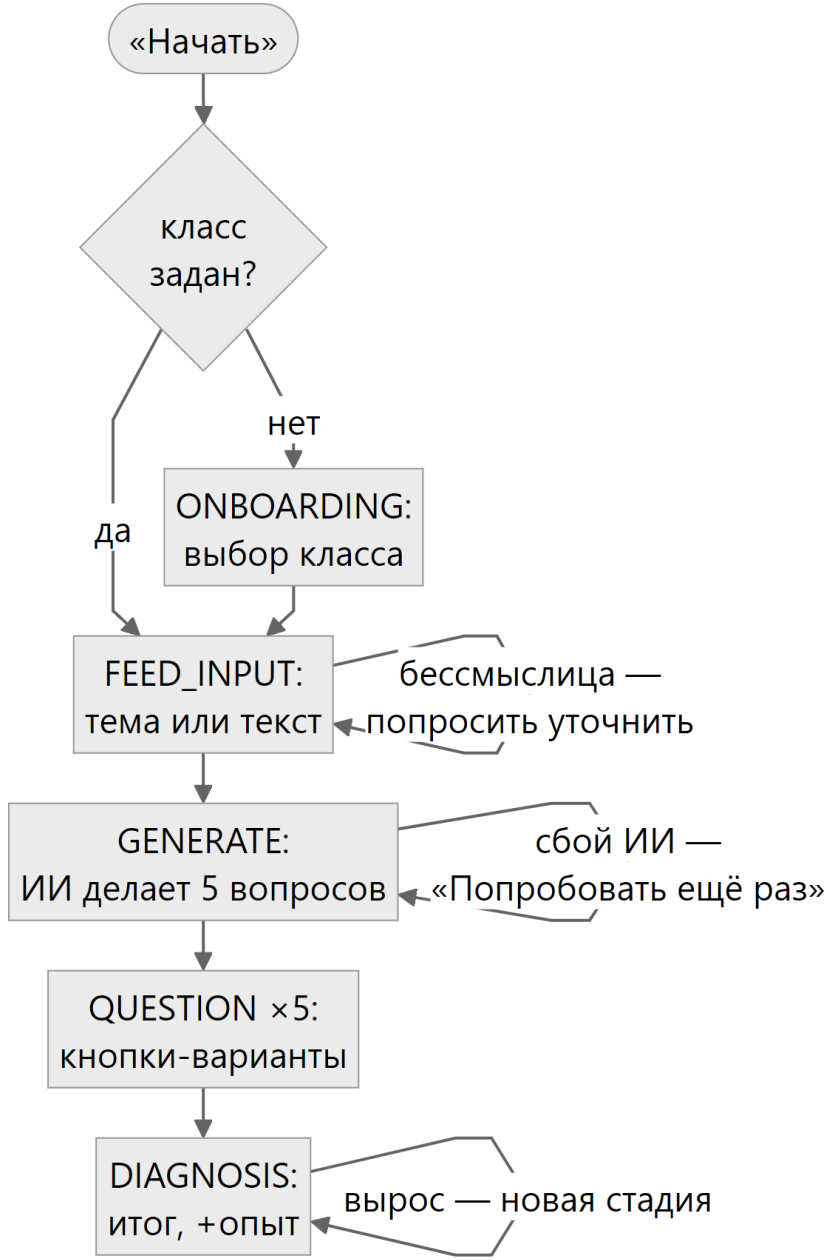
[FOLLOWUP]  — промт followup: 2 вопроса только по слабой подтеме (тот же цикл, что QUESTION)
  Оба верно → +30 опыта, подтема удаляется из weak, добавляется в closed,
              «🥕 Тема закрыта! Грызлик объелся!»
  Иначе     → +5 опыта, «Почти! Вернёмся к этому через пару дней»

[STATS]  — команда /stats или кнопка «📈 Мой прогресс»
  Уровень, имя уровня, опыт до следующего уровня, сытость, серия,
  количество слабых мест, последние закрытые темы

[WEAK_LIST]
  Список слабых подтем (до 8), кнопка на каждую → [EXPLAIN_MENU]
```

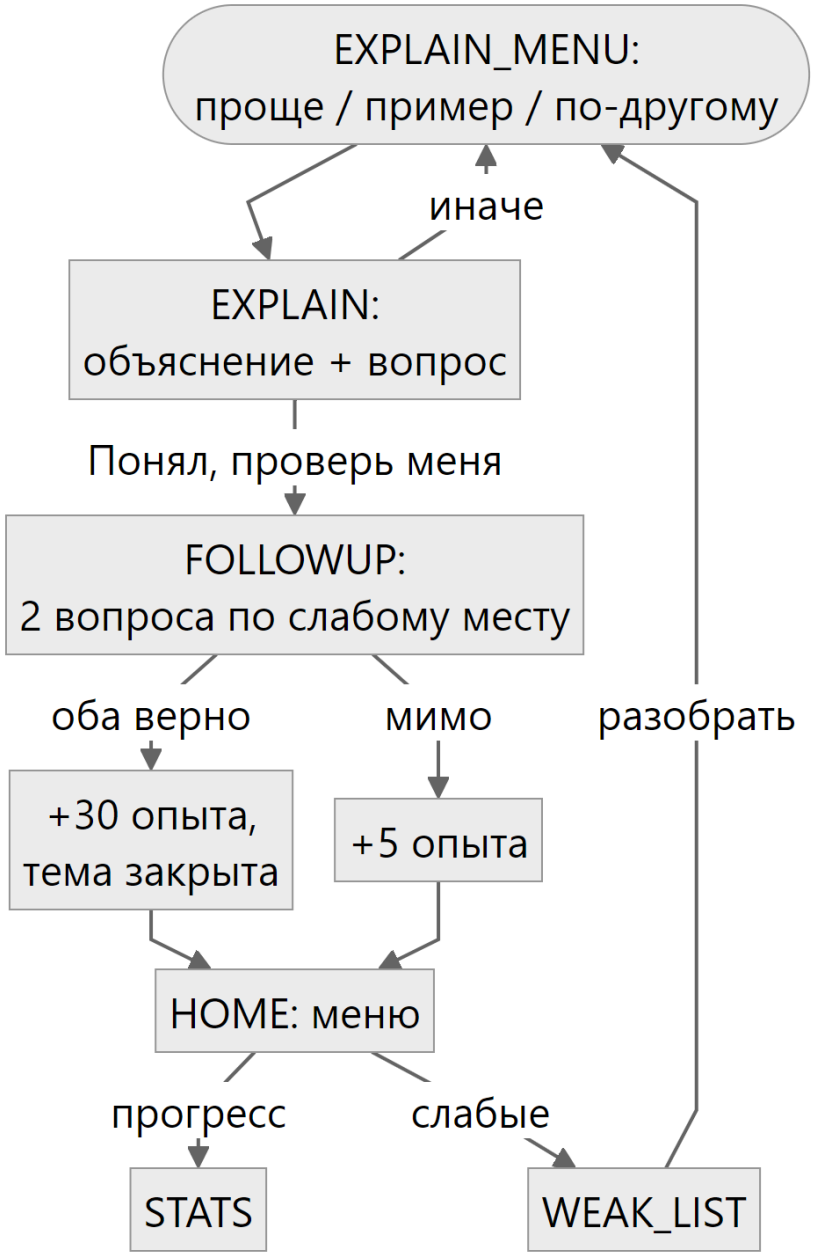
Почему граница 300 символов: короткий ввод — обычно название темы («Дроби, 6 класс»), и по нему ИИ опирается на школьную программу. Всё, что длиннее, уже похоже на настоящий параграф — тогда ИИ отвечает только по фактам из текста, без выдумок.

3. Схема: онбординг → тест → диагностика

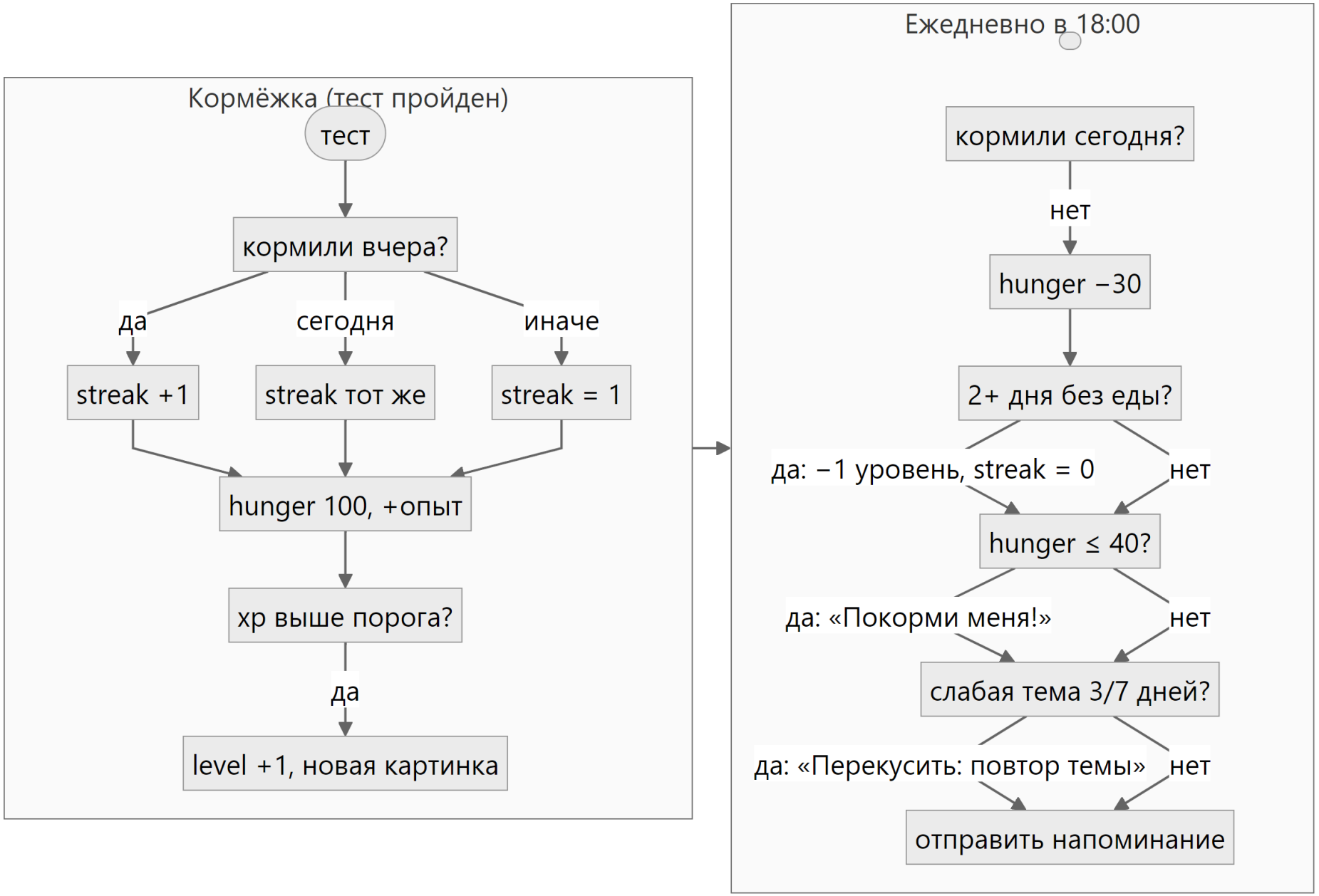


Что происходит на стрелках и в ошибках — см. раздел 2, там всё по шагам.

4. Схема: объяснение → добивающие → меню



5. Схема: логика питомца



6. Уровни и картинки

Уровень	Имя	Порог опыта	Картинка
1	Яйцо	0	`pet_1.png`
2	Малыш	150	`pet_2.png`
3	Подросток	450	`pet_3.png`
4	Взрослый	900	`pet_4.png`
5	Мудрец	1800	`pet_5.png`

Приветствие и опрос класса — с `mascot.png`.

7. Детали MAX Bot API (реализовано в bot.py)

- Базовый URL `https://platform-api2.max.ru`, токен только в заголовке `Authorization`.
- Long polling `GET /updates` с `marker`, `timeout=30`, `types=bot_started,message_created,message_callback`, пауза ≥ 300 мс между запросами.
- Отправка: `POST /messages?chat_id=…`, тело `{text, attachments}`; текст ≤ 4000 символов, длинный — режется на части по абзацам; кнопки — attachment `inline_keyboard` (`type: callback`, payload ≤ 1024); картинка — attachment `image` с токеном из `POST /uploads?type=image`.
- Нажатие кнопки — `message_callback` → обязательный ответ `POST /answers?callback_id=…` (попутно у исходного сообщения снимается клавиатура через `answer.message`).
- Загрузка картинок асинхронная: при `attachment.not.ready` отправка повторяется с паузой.
- TLS: корневой сертификат Минцифры подгружается в SSL-контекст (`certs/`), проверка не отключается.
- Дедупликация апдейтов в БД (`seen`) — повторная доставка не дублирует опыт.
- Апдейты одного ученика обрабатываются строго по очереди (asyncio.Lock на пользователя).